

Reliability-Aware Fault Injection for Microscaling-Based DNNs

Peeyush Agarwal

CS496 Undergraduate Project, Indian Institute of Technology Kanpur

5 October 2026

Abstract

Microscaling (MX) formats cut the cost of low-precision inference in deep neural network (DNN) accelerators, but how they behave under hardware faults has barely been studied. In an MX format a block of values shares one scale. So the effect of a bit fault depends not only on the value it corrupts but also on the shared scale and on the other values in the block. This report presents a reliability analysis methodology for MX-based DNNs and the open-source fault-injection framework, `mxfi`, that implements it. The methodology injects faults into the real MX bit encoding, at element codes and at shared scales. It measures fault sensitivity across five element formats, four block sizes, two tensor types and every bit and layer position. The study covers convolutional, re-parameterised and Transformer models on CIFAR-10 and CIFAR-100. Two exhaustive campaigns, about 1.1 million injections, give exact failure rates that validate the sampled estimates. Three results stand out. First, formats with matching accuracy differ in vulnerability by up to three orders of magnitude, and the ordering $e3m2 < e4m3 < e5m2$ holds on every network measured. Second, the main cause is the NaN and infinity codes of the 8-bit formats: decoding those codes as zero on read cuts element-fault damage by at least $29\times$ on the larger models. Third, a fault in a shared scale is always more damaging than a fault in an element, typically by $13\times$ to more than $1000\times$. The study also shows that the usual “any image changed” SDC metric can hide real differences between formats, and it settles two open design choices for MX fault injection.

1 Introduction

DNN accelerators are moving to narrow number formats because memory traffic and arithmetic both get cheaper as bit width falls. A 4- to 8-bit value cannot cover the range of values in a weight or activation tensor on its own. *Microscaling* (MX) formats solve this by grouping values into small blocks that share one scale factor [1, 2]. The OCP MX specification defines 8-, 6- and 4-bit floating-point variants (MXFP8, MXFP6, MXFP4), and they are now supported in commercial hardware.

Hardware is not fault-free. Radiation strikes, voltage droop and ageing can flip stored bits, and in a DNN such a flip can silently change a prediction. This kind of error is called *silent data corruption* (SDC). DNN reliability under bit flips is well studied for FP32 and integer formats [3, 4, 5], and bit flips have also been used as an attack [6, 7]. These studies all assume that each value is a self-contained word, so one flip changes one number. MX breaks this assumption. A flip in an element code still changes one value, but a flip in the shared scale changes every value in the block at once. How much this matters, and which properties of a format make a network more or less fragile, had not been measured.

Objectives. The project set out five objectives:

- O1 Fault propagation:** characterise how single-bit faults in elements and in shared scales propagate, and quantify the difference.
- O2 Format sensitivity:** measure sensitivity across MXFP8, MXFP6 and MXFP4, and test whether reliability differs at comparable accuracy.
- O3 Block size:** determine how the scale-sharing granularity trades efficiency against reliability.
- O4 Tensor and location:** compare weights with activations, scale bits with element bits, high-order with low-order bits, and layer with layer.
- O5 Vulnerability drivers:** identify the representation characteristics that explain the observed vulnerability.

Contributions.

- **mxfi**, a bit-accurate fault-injection framework for MX formats. It encodes tensors into real MX bit patterns, injects faults at either site, and supports sampled, stratified and exhaustive campaigns for weights and activations.
- A characterisation across all four representation axes on three architecture families and two datasets, with more than sixty independently trained networks. Two exhaustive campaigns give ground truth.
- An explanation of the observed vulnerability. Special (NaN/Inf) codes dominate element damage and can be neutralised with a simple read-side guard. The remaining perturbation damage is governed by the decision margins of the network, not by mantissa width.
- Four methodological findings for anyone running such campaigns. The common “any image changed” SDC rate is confounded by the evaluation set. The severity measure in the original plan does not predict failures, and a gradient-based replacement cuts the required injections by 57–120×. Single-bit injection is sufficient. Exact enumeration over the MX code table outperforms learned value intervals.

Section 2 explains MX formats and the fault model. Section 3 describes the methodology and Section 4 the experimental setup. Section 5 presents the results objective by objective. Section 6 covers the methodological findings, Section 7 gives design guidance, Section 8 discusses limitations, and Section 9 concludes.

2 Background

2.1 Microscaling formats

An MX block holds K values (32 in the specification). It is stored as one 8-bit shared exponent X , called the *scale*, together with K narrow element codes m_i . The hardware reconstructs each value as

$$v_i = \text{decode}(m_i) \cdot 2^{X-127}, \quad i = 1, \dots, K. \quad (1)$$

The scale uses the E8M0 encoding: an unsigned power of two, with code 255 reserved for NaN. Each element is a small sign–exponent–mantissa number. For a nonzero exponent field e it decodes to $(-1)^s (1 + f/2^M) 2^{e-\text{bias}}$, where f is the M -bit mantissa field. Table 1 lists the five element formats studied.

Table 1: MX element formats. The two 8-bit formats reserve codes for NaN or infinity; the 6- and 4-bit formats do not, so every one of their bit patterns is a finite number.

format	family	sign/exp./mantissa bits	bias	largest value	NaN/Inf codes
e4m3	MXFP8	1/4/3	7	448	2 (NaN)
e5m2	MXFP8	1/5/2	15	57,344	8 (2 Inf, 6 NaN)
e3m2	MXFP6	1/3/2	3	28	none
e2m3	MXFP6	1/2/3	1	7.5	none
e2m1	MXFP4	1/2/1	1	6	none

The OCP rule chooses the scale so that the largest value in the block lands in the top binade of the element format: $X = \lfloor \log_2 \max_i |v_i| \rfloor - e_{\max}$, where $e_{\max}$ is the exponent of the largest representable element. The largest representable element lies below the top of that binade, so the block maximum can be clipped by up to 12.5% (e4m3, e5m2, e3m2), 6.25% (e2m3) or 25% (e2m1). As a control, **mxfi** also implements a non-clipping rule that picks the smallest scale at which the block maximum is representable, $X = \lceil \log_2(\max_i |v_i|/v_{\max}) \rceil$. Results use the OCP rule unless stated otherwise.

2.2 Faults in an MX block

In ordinary floating point a bit flip perturbs exactly one number. Under Equation (1) there are two structurally different fault sites (Figure 1):

- An **element fault** flips a bit of one code m_i . A mantissa bit changes the value by a small relative amount, an exponent bit multiplies it by a power of two, and the sign bit negates it. In e4m3 and e5m2 a flip can also turn the code into NaN or infinity.
- A **scale fault** flips bit j of X . This shifts the exponent by $\pm 2^j$, so all K values in the block are multiplied by $2^{\pm 2^j}$ at once. A flip that produces code 255 turns the whole block into NaN.

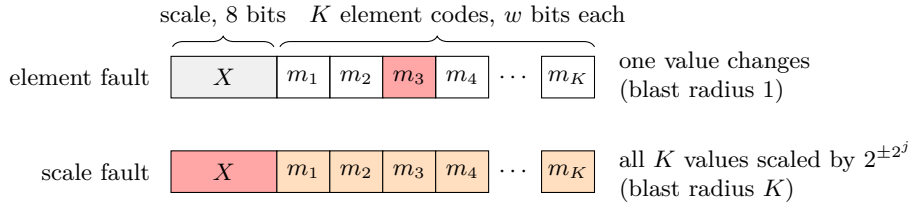


Figure 1: One MX block and its two fault sites. Red marks the flipped word and orange the values it alters.

The number of values a fault alters is its *blast radius*: 1 for an element fault and K for a scale fault.

A simple calculation predicts that the scale matters before any measurement is made. A block occupies $8 + Kw$ bits, of which only 8 belong to the scale: about 3% for $K = 32$ and $w = 8$. If every stored bit is equally likely to flip, the expected number of corrupted *values* per block is $8K$ from scale faults and Kw from element faults, a ratio of $8 : w$. An 8-bit format therefore receives about half of its corrupted values through the 3% of storage that holds scales, and a 4-bit format about two thirds.

2.3 Related work

Fault-injection studies of DNNs have shown that errors propagate differently by layer, data type and bit position [3], and frameworks such as Ares [4] and PyTorchFI [5] made such campaigns routine for FP32 and integer models. Bit-flip attacks [6, 7] show that a few well-chosen flips can destroy accuracy. Statistical fault injection bounds the error of a sampled campaign [8]. TreeFI [9] reduces campaign cost for FP32 by learning value intervals with similar bit-flip behaviour and allocating injections across them. These works target FP32 and integer formats and do not model a shared scale. The original plan for this project asked whether TreeFI’s interval learning carries over to MX. Section 6.4 answers that question.

3 Methodology

3.1 Fault model

I use the single-bit-flip model that is standard in DNN fault-injection work, applied to stored values: one bit of one element code, or of one scale byte, is inverted. Section 6.4 tests this choice against double-bit faults. Weight faults are *persistent*: a corrupted weight is read by every inference until the word is rewritten. Activation faults are *transient*: they affect only the inference in which they occur. Unless stated otherwise, every stored bit is equally likely to be hit, so faults land in proportion to storage.

3.2 Metrics

Let $D = \{x_1, \dots, x_N\}$ be a fixed evaluation set, $\hat{y}_i$ the prediction of the fault-free, MX-quantised network (the *golden* prediction), and $\hat{y}_i^\varphi$ the prediction under fault φ . Table 2 defines the metrics.

Table 2: Metrics. The per-inference rate is the primary metric of this report.

metric	definition	meaning
corruption count	$c(\varphi) = \sum_i \mathbb{1}[\hat{y}_i^\varphi \neq \hat{y}_i]$	inferences the fault changed
per-inference rate	$r(\varphi) = c(\varphi)/N$, averaged over faults	chance one inference is corrupted
any-image SDC rate	fraction of faults with $c(\varphi) \geq 1$	did the fault change anything at all
non-finite rate	fraction of faults giving a NaN/Inf output	faults that destroy the output

The **per-inference rate** answers the question a system designer cares about: given one fault, what is the probability that a particular inference returns a wrong answer? It is also the only quantity on which weight and activation faults can be compared, because a weight fault is exposed to all N inferences and an activation fault to one. The **any-image SDC rate** is the conventional metric, but Section 6.1 shows that it mostly measures whether the evaluation set contains an input near a decision boundary. It is therefore reported only where it adds something. Proportions carry Wilson 95% intervals [15], per-inference rates carry normal intervals on the mean over faults, and activation rates carry bootstrap intervals over images.

3.3 The injection engine

The campaign compares a golden run with a faulty one (Algorithm 1). Three design decisions keep the injection faithful to what an accelerator executes.

1. **Deployment form.** Batch normalisation is folded into the preceding convolution and RepVGG’s branches are fused before quantisation. An unfolded batch normalisation would re-normalise whatever the corrupted convolution produced and hide part of the damage.
2. **Code-domain injection.** Tensors are encoded into real MX bit patterns, the fault is applied to the stored code, and the block is decoded only afterwards. This is what allows a scale fault to perturb all K values, and an element fault to land on a NaN code.
3. **Faithful special values.** The OCP conversion takes the scale from the block maximum, so a NaN anywhere in a block makes the whole block NaN. `mxfi` reproduces this. An earlier version of the codec mapped NaN inputs to zero, which silently erased NaNs created by activation faults at the next layer. All activation results in this report were re-measured after fixing it, and a unit test now pins the behaviour.

Activation faults are injected through forward hooks that quantise a layer’s input to MX, flip one bit of one image’s encoding, and decode. Activation blocks run along the channel axis, so each image’s blocks are independent and a batch of one is the exact per-inference fault space.

Algorithm 1 Weight fault-injection campaign

Require: network M , format F , block size K , evaluation set D , budget n

```

1:  $M \leftarrow \text{DEPLOY}(M)$  ▷ fold batch norm, fuse branches
2: for all quantisable layers  $\ell$  do
3:   view  $W_\ell$  as (outputs  $\times$  reduction); split the reduction axis into blocks of  $K$ 
4:    $X_\ell \leftarrow$  per-block scale (OCP rule);  $C_\ell \leftarrow \text{round}_F(W_\ell 2^{-X_\ell})$ 
5:    $W_\ell \leftarrow \text{DECODE}(X_\ell, C_\ell)$  ▷ the network now holds MX values
6: end for
7:  $\hat{y} \leftarrow \arg \max M(D)$  ▷ golden predictions
8: for  $t = 1$  to  $n$  do
9:   draw  $\varphi = (\ell, \text{site}, \text{index}, b)$  uniformly over all stored bits
10:  if site = element then  $C_\ell[\text{index}] \leftarrow C_\ell[\text{index}] \oplus 2^b$ ; re-decode that one weight
11:  else  $X_\ell[\text{index}] \leftarrow X_\ell[\text{index}] \oplus 2^b$ ; re-decode the  $K$  weights of the block
12:  end if
13:   $\hat{y}^\varphi \leftarrow \arg \max M(D)$ ; record  $c(\varphi)$ , which images changed, non-finiteness; restore
14: end for
15: return rates over all faults with confidence intervals

```

3.4 Exhaustive and sampled campaigns

Line 9 of Algorithm 1 defines the sampling. Replacing it with an enumeration of every addressable bit makes the campaign *exhaustive*, which gives the true failure rate with no sampling error. MX bit spaces are small enough that this is affordable for ResNet8: 638,624 injections under e4m3 and 483,904 under e3m2. These two campaigns serve as ground truth for validating the sampler and for testing the sampling strategies in Section 6. Element alphabets hold at most 256 codes, so the decoded value of every (code, bit) pair can also be tabulated exactly.

4 Experimental setup

Table 3 lists the networks. ResNet8 is the MLPerf Tiny image-classification model [11, 10]. RepVGG-A0 [12] is a re-parameterised network that is fused into plain 3×3 convolutions for deployment. The vision transformer [13] uses the embedding width and head count of DeiT-Tiny [14], depth 6 and 4×4 patches. Every weight matrix is a linear layer, so MX blocks run along the model dimension, and LayerNorm is left unquantised, as in deployment. To separate the effect of model capacity, ResNet8 was also trained at five widths spanning a 35-fold range of parameter counts, with ten independently trained networks at each width.

Campaign parameters. Formats e4m3, e5m2, e3m2, e2m3 and e2m1; block sizes $K \in \{8, 16, 32, 64\}$ with $K = 32$ as the default. The evaluation set is a fixed, class-balanced subset of 200 test images (2 per class on

Table 3: Networks. Accuracies are FP32 top-1. All networks were trained from scratch.

network	parameters	CIFAR-10 acc.	seeds	CIFAR-100 acc.	seeds
ResNet8	78k	87.0%	1 + width sweep	58%	3
ResNet8 width sweep	20k–692k	varies	10 per width	–	–
RepVGG-A0 (fused)	7.0M	91.4–91.5%	3	70–71%	3
ViT (DeiT-Tiny width)	2.7M	80.4–81.9%	3	47–49%	3

CIFAR-100). Sampled campaigns use 3000 injections unless stated otherwise. In total the study recorded about 1.44 million sampled faults in 119 weight campaigns on CIFAR-10, the two exhaustive campaigns (1.12 million injections), 390,000 multi-bit injections, and further campaigns for activations and CIFAR-100.

Datasets. The original plan included DeiT on ImageNet. ImageNet was not available on any machine this project could access, so CIFAR-100 stands in for it: ten times the classes and a tenth of the images per class, so networks sit much closer to their decision boundaries. Section 8 discusses what this substitution does and does not show.

Compute. Early campaigns ran on a 4-core laptop CPU. The width sweep, the CIFAR-100 study and the larger campaigns ran on an NVIDIA A100 GPU server at IIT Kanpur. Training is not bit-reproducible across machines, so every campaign is tied to the exact checkpoint it was run on, and this is checked by replaying recorded faults against the checkpoint.

5 Results

5.1 Validating the sampler against exhaustive ground truth

Under e4m3, the exhaustive campaign on ResNet8 gives a true any-image SDC rate of 0.4171. An ordinary 3000-injection campaign returned 0.4270 with 95% interval $[0.4094, 0.4448]$: within 2.4% of the truth, with the interval containing it. Under e3m2 the exhaustive rate is 0.1079 and the sampled estimate 0.1070 $[0.0964, 0.1186]$. The stratified per-inference estimator used in Section 5.3 agrees as closely: a sampled estimate of 0.016357 against the exhaustive 0.016354. The two exhaustive campaigns were recorded on two different trained copies of ResNet8 of equal accuracy, so they are used as two independent references and never as a comparison between formats.

5.2 O2: Equal accuracy does not mean equal reliability

Table 4 gives the per-inference rate of element faults for the three formats that bracket the design space. On the transformer, e5m2 and e3m2 differ in accuracy by 0.01 points (81.74% against 81.73%), yet a single element fault in e5m2 is about 2000 times as likely to corrupt a given inference. On ResNet8 the three formats lie within 0.3 points of each other (84.8–85.1%) and still differ by more than an order of magnitude. The ordering $e3m2 < e4m3 < e5m2$ holds on all eight network configurations, with disjoint 95% intervals throughout. It also holds on all nine CIFAR-100 networks (Section 5.9).

Table 4: Per-inference rate of element faults, $K = 32$, averaged over seeds. Lower is better. The ordering $e3m2 < e4m3 < e5m2$ holds on every row, with disjoint intervals.

network	seeds	e3m2 (6-bit)	e4m3 (8-bit)	e5m2 (8-bit)
RepVGG-A0	3	0.00001	0.00688	0.05381
ViT	3	0.00003	0.01258	0.06897
ResNet8 (reference)	1	0.00332	0.01699	0.07801
ResNet8, width 8	5	0.01003	0.02005	0.08143
ResNet8, width 16	5	0.00180	0.01374	0.07831
ResNet8, width 24	4	0.00121	0.01210	0.07585
ResNet8, width 32	4	0.00060	0.01354	0.07934
ResNet8, width 48	4	0.00035	0.01302	0.07812

Two independent mechanisms produce these gaps: special codes (Section 5.3) and network sensitivity (Section 5.4).

5.3 O5: Special codes are the dominant driver

Only formats that own NaN or infinity codes can produce a non-finite value from an element flip, and nothing downstream absorbs a NaN: it propagates through every later layer and destroys the output. On the transformer 94% of e5m2’s element failures were non-finite events, and on RepVGG 92%. No e3m2 element fault produced one in any campaign, including the 483,904-fault exhaustive enumeration, as having no special codes requires. Measured by damage rather than count, non-finite events carry 96.9% of e5m2’s per-inference element damage. In e4m3 only 1.27% of element faults turn a code into NaN, but each of them corrupts every inference, so together they carry 78% of all element damage on ResNet8.

Capacity absorbs perturbations, not NaNs. The width sweep isolates this mechanism (Figure 2). The non-finite rate is a property of the format: it varies by less than 20% across the 35-fold range of capacity and does not correlate with model size. What changes with capacity is how well the network absorbs ordinary perturbations. Over that range the format with no special codes (e3m2) improves 25 $\times$, the format with two (e4m3) improves 1.5 $\times$, and the format with eight (e5m2) does not improve measurably (0.96 $\times$). The benefit a format draws from a larger model falls in exact order of its number of special codes.

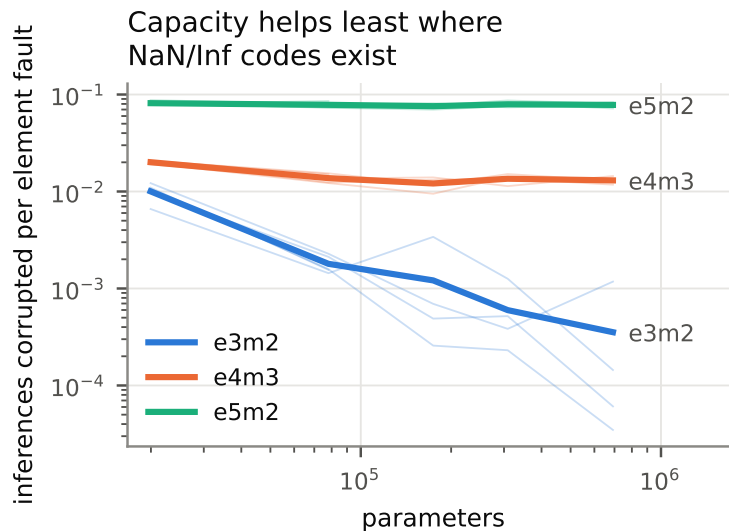


Figure 2: Per-inference rate of element faults against parameter count for ResNet8 at five widths. Each faint line is one independently trained network; bold lines are means. Capacity helps least for the format with the most NaN/Inf codes, and not at all for e5m2.

A read-side guard. If special codes are the problem, a cheap fix follows: decode any special element code as zero when it is read. A fault that would have produced NaN then merely zeroes one weight. I tested this guard on all three architectures under e4m3 and e5m2 (Table 5). Every element fault that lands on a special code was found by enumeration and injected with and without the guard. On RepVGG and the transformer, where these faults carry 97–100% of element damage, the guard cuts the element per-inference rate by at least 29 $\times$, and by up to 4100 $\times$. This brings e5m2 down to the level of e3m2. On ResNet8 the reduction is smaller (3.9–53 $\times$), because a small network is also fragile to ordinary perturbations, which the guard does not touch. No guarded fault produced a non-finite output. The guard was simulated in software; its hardware cost is one comparator on the read path.

5.4 O5: Sensitivity, and what it really measures

Special codes do not explain everything: formats without them still differ. To separate the network from the encoding, I perturbed single weights by 1.5 times the layer’s RMS value, a probe that involves no bit patterns at all. In 1500 trials it never changed a prediction of the e3m2 or e5m2 networks, but changed e4m3’s in 2–5% of trials. The fragility belongs to the quantised network, not to the encoding.

The responsible quantity is the sensitivity $s_i = |\partial m / \partial w_i| \cdot \text{rms} / m$, where m is the margin between the top two logits. A perturbation of $\delta \cdot \text{rms}$ flips a prediction roughly when $s_i \delta > 1$. Computed by backpropagation, s reproduces the probe with correlation +0.998, and across the five formats it predicts the perturbation-driven

Table 5: Effect of decoding special element codes as zero on read (CIFAR-10). The reduction is the unguarded rate divided by a 95% upper bound on the guarded rate, so it is a lower bound.

network	format	share of element damage from special codes	reduction, at least
ResNet8 (2 nets)	e4m3	78–93%	3.9–10.7×
ResNet8 (2 nets)	e5m2	98–99%	30–53×
RepVGG-A0 (3 seeds)	e4m3	99.5–99.9%	115–389×
RepVGG-A0 (3 seeds)	e5m2	99.9–100%	319–4112×
ViT (3 seeds)	e4m3	97.1–99.5%	29–133×
ViT (3 seeds)	e5m2	99.9–100%	43–2479×

failure rate at $r = +0.93$. Mantissa width does not: it correlates at -0.25 , and e2m1, with the fewest mantissa bits, has the highest perturbation-driven failure rate on CIFAR-10.

Taken further, s measures something narrower than it appears to. Its gradient factor agrees across all five formats to within $1.08\times$. The whole $10\times$ spread in s comes from the margin in its denominator, specifically from the single evaluation image closest to a decision boundary. That image belongs to the evaluation set rather than to the network: $1/\min m$ ranks fifteen networks by failure rate at Spearman $+0.98$ on the campaign’s own 200 images, but at only $+0.21$ on 2000 fresh images. This finding leads directly to the metric problem in Section 6.1.

5.5 O1: Shared scales dominate

Scale faults outweigh element faults on every architecture, format and dataset tested (Table 6). In the exact ResNet8 enumeration under e4m3, 94.4% of all possible scale flips change at least one prediction, and a scale fault corrupts on average 49.9 of 200 inferences against 3.3 for an element fault: a $15\times$ difference per inference. On the larger models the ratio reaches the order of 10^3 for e3m2. These large ratios are lower bounds, since the e3m2 element rate there is statistically indistinguishable from zero. The ratio shrinks only for e5m2 ($2.6\times$), because that format’s element faults already corrupt everything through the NaN pathway.

Table 6: Scale-to-element damage ratio, per inference. The direction never reverses.

network	format	scale / element
ResNet8 (exhaustive)	e4m3	15.3×
ResNet8, width 16	e4m3	13.3×
ResNet8, width 16	e5m2	2.6×
ResNet8, width 48	e3m2	434×
ViT	e3m2	order 10^3 (lower bound)
RepVGG-A0	e3m2	order 10^3 (lower bound)

Scale faults also produce non-finite outputs in 7–13% of cases *for every format*, including those with no special element codes. The E8M0 scale has its own NaN code, and its most significant bit is worth 128 exponent steps, enough to overflow any value. Protecting the scale is therefore necessary even for a format like e3m2.

5.6 O4: Fault location by bit and by layer

Bit position. Figure 3 shows the exhaustive per-bit damage for ResNet8 under e4m3. Among faults that leave the output finite, the sign bit does the most damage of any element bit. In total damage, however, the sign bit ranks seventh of eight, because the dominant damage comes from flips into the NaN code and a sign flip is the one flip that can never produce it. In e3m2, which has no NaN code, the sign bit is the worst bit outright. In the scale, the picture is set by which bits are normally set. On this network the scale bytes lie between 114 and 120, so bits 4–6 are always 1 and bit 3 is almost always 0. Flipping bit 3 multiplies the block by 256 and corrupts 75% of inferences; flipping bit 7 overflows it and corrupts 98%. Flipping bits 4–6 shrinks the block towards zero and corrupts only 3%. Growing a block is catastrophic and shrinking it is mild.

Layer. Vulnerability runs opposite to layer size (Figure 4). In the exact enumeration the 3584-bit stem convolution is the most fragile layer (any-image SDC 0.853), because its errors propagate through the whole network. The 304,128-bit final convolution is the most robust (0.289). Since a uniform campaign samples

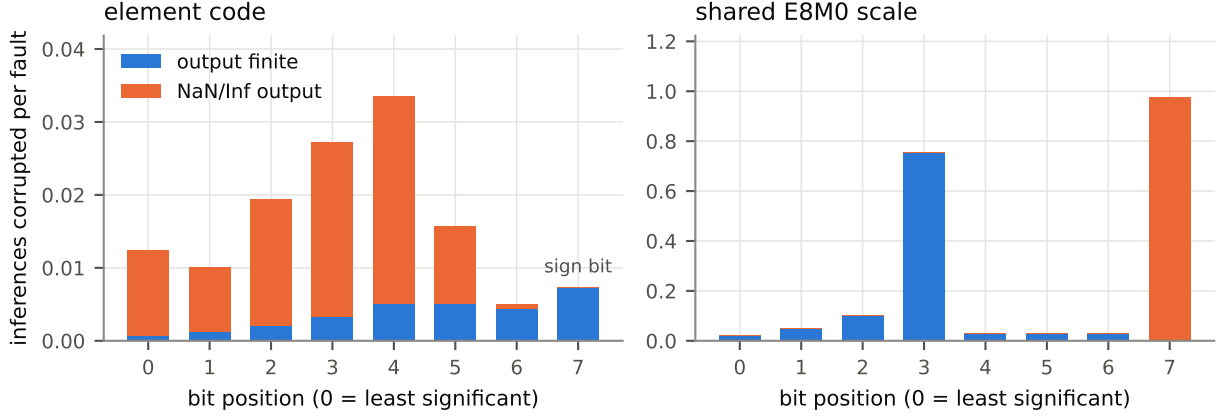


Figure 3: Exhaustive per-bit damage per inference (ResNet8, e4m3, $K = 32$, 638,624 injections), split by whether the output stayed finite. The sign bit does the most finite damage, but most element damage comes from flips into the NaN code, which a sign flip cannot make. Scale bits 3 and 7 are almost never set, so flipping them grows the block $256\times$ or overflows it; bits 4–6 are always set, and flipping them shrinks it.

layers in proportion to their size, it spends 48% of its budget on the most robust layer and 0.6% on the most fragile. Protection should be allocated by vulnerability, not by size.

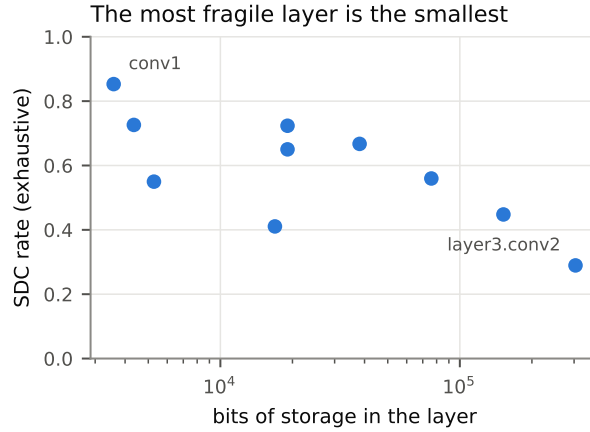


Figure 4: Exhaustive any-image SDC rate of each ResNet8 layer against its storage (e4m3). The smallest layer, the stem, is the most fragile.

5.7 O4: Weights and activations

Compared on the per-inference rate, an element fault in a weight does $0.6\text{--}2.2\times$ the damage of one in an activation, and a scale fault $0.9\text{--}1.7\times$ (Table 7). Under e4m3 the two are statistically indistinguishable on CIFAR-10. Under e5m2 weights are $1.4\text{--}2.2\times$ worse, because a smaller share of activation faults reaches a special code ($3.4\text{--}4.7\%$, against $5.4\text{--}7.7\%$ for weights). The reason is the same in both tensor types: about 1% of faults land on a special code and destroy the inference. The real difference between weights and activations is *exposure*, not per-fault vulnerability. A weight fault persists across every inference until the memory is rewritten, while an activation fault lives for one inference. The shared scale dominates in both: scale faults are $14\times$ worse than element faults in activations and $11\times$ in weights, on ResNet8.

5.8 O3: Block size

Block size is nearly free in accuracy: on ResNet8, across $K = 8$ to 64, every format moves by about one point or less. That makes it a clean reliability axis, and the two fault sites move in opposite directions along it (Table 8). Raising K from 8 to 64 cuts the share of storage held by scales from 11.1% to 1.7%, so a scale is less likely to be hit. But each scale fault becomes more damaging, because it now affects more values: its any-image SDC rate rises from 0.73 to 0.94. Element faults, meanwhile, corrupt fewer inferences as K grows

Table 7: Per-inference rate of element faults in weights and activations (CIFAR-10, $K = 32$). Activation intervals are bootstrapped over 100 images.

network	format	weight	activation [95% CI]	weight / activation
ResNet8	e4m3	0.0170	0.0122 [0.0077, 0.0182]	1.4×
RepVGG-A0	e4m3	0.0048	0.0075 [0.0049, 0.0101]	0.64×
ViT	e4m3	0.0122	0.0114 [0.0080, 0.0150]	1.07×
ResNet8	e5m2	0.0780	0.0359 [0.0302, 0.0421]	2.2×
RepVGG-A0	e5m2	0.0549	0.0378 [0.0319, 0.0442]	1.45×
ViT	e5m2	0.0659	0.0486 [0.0420, 0.0554]	1.36×

(1.8× fewer on ResNet8 and 5.8× on RepVGG under e4m3). The trend is monotone and present under both scale rules, but this study does not establish its mechanism.

Table 8: Block size on ResNet8 (e4m3). Larger blocks shrink the scale’s share of storage but make each scale fault worse; element faults become milder.

	$K = 8$	$K = 16$	$K = 32$	$K = 64$
scale bits, share of storage	11.1%	5.9%	3.1%	1.7%
scale fault, any-image SDC	0.731	0.890	0.924	0.941
scale fault, inferences corrupted (of 200)	40.1	45.3	48.7	50.1
element fault, per-inference rate	0.0241	0.0208	0.0159	0.0136

The choice of block size is therefore a genuine trade. Small blocks localise scale damage at the cost of more scale storage; large blocks save storage but concentrate more of the risk in fewer, more damaging scale bytes. A designer who protects the scales (for example with parity or ECC) can use large blocks safely, since the scale is then a small amount of protected storage.

5.9 A harder task: CIFAR-100

All three architectures were retrained on CIFAR-100 with unchanged training recipes. Every headline result replicates. Formats are matched in accuracy (four formats within 0.01 points of each other on the ViT) and the ordering $\text{e3m2} < \text{e4m3} < \text{e5m2}$ holds on all nine networks. Scale faults outweigh element faults everywhere, from 1.7× to 3600×. NaN codes carry 94–100% of e4m3 and e5m2 element damage on the larger models.

What changes is the floor (Table 9). A harder task leaves the networks with smaller decision margins, so the formats without special codes become 3–22× more vulnerable, while e4m3 and e5m2 barely move, because a NaN corrupts an inference however wide the margins are. The gap between formats therefore narrows: on the ViT the e4m3/e3m2 ratio falls from 461× to 19×. The advantage of NaN-free formats is still large, but it is smallest on exactly the harder tasks real deployments face. The read-side guard still works on CIFAR-100, with reductions of at least 13–1387× on RepVGG and the ViT. Among the NaN-free formats the ordering does not carry over: on CIFAR-100 the ViT ranks e2m1 as the safest of the three. This is consistent with Section 5.4: perturbation damage follows network sensitivity, not field widths.

Table 9: Per-inference rate of element faults, CIFAR-10 → CIFAR-100, mean over seeds.

network	e3m2 (no special codes)	e4m3	e5m2	e4m3/e3m2 gap
ViT	$2.7\text{e-}5 \rightarrow 6.1\text{e-}4$ (22×	$0.0126 \rightarrow 0.0114$	$0.069 \rightarrow 0.077$	$461\times \rightarrow 19\times$
RepVGG-A0	$1.4\text{e-}5 \rightarrow 5.6\text{e-}5$ (3.9×	$0.0069 \rightarrow 0.0081$	$0.054 \rightarrow 0.057$	$476\times \rightarrow 144\times$
ResNet8	$1.8\text{e-}3 \rightarrow 5.7\text{e-}3$ (3.2×	$0.0137 \rightarrow 0.0169$	$0.078 \rightarrow 0.082$	$7.6\times \rightarrow 3.0\times$

6 Methodological findings

6.1 The usual SDC metric hides the effect it is used to measure

A fault is conventionally counted as causing SDC if *any* of the N evaluated inferences changes its prediction. This quantity grows with N and saturates, so beyond a point it measures whether the evaluation set happens

to contain an input near a decision boundary rather than anything about the fault. To test this, I replayed one fixed list of 1500 element faults against nine class-balanced sets of 200 images, on each of two independently trained ResNet8 networks. The any-image rate moved by up to 7 times the width of its own confidence interval from one image set to the next, and it tracked each set’s smallest margin (Spearman +0.80 to +0.93). Under this metric e3m2 and e4m3 looked equal ($1.03\times$ and $0.93\times$), and their order reversed between image sets. Scored per inference, the same faults on the same image sets gave $5.6\times$ and $7.6\times$ between the formats, stable to within $1.15\text{--}1.27\times$ across image sets.

The effect was real, and the conventional metric was destroying it. Several earlier conclusions in this project, including an apparent reversal of the format ranking between architectures, turned out to be artefacts of this metric and were withdrawn. The per-inference rate should be the primary metric in MX fault-injection studies.

6.2 A severity measure that predicts failure

The original plan scored an element fault by $d = |\log_2 |v'| - \log_2 |v||$, the change in log-magnitude. Over 39,337 element faults its rank correlation with observed failure is -0.035 and its AUC is 0.435, slightly worse than chance (Figure 5). The reason is in the definition: negating a value leaves $|v|$ unchanged, so a sign flip scores zero severity even though the sign bit does more perturbation damage than any other element bit. Three better measures follow in order: relative error (AUC 0.699), absolute change in layer-RMS units (0.845), and

$$\text{margin_shift}_i = s_i \frac{|v'_i - v_i|}{\text{rms}},$$

the fraction of the decision margin a fault consumes, with AUC 0.990. Its threshold needs no tuning: flagging faults with $\text{margin_shift} > 1$ catches 98.5% of all failures while flagging only 4.4% of the fault space.

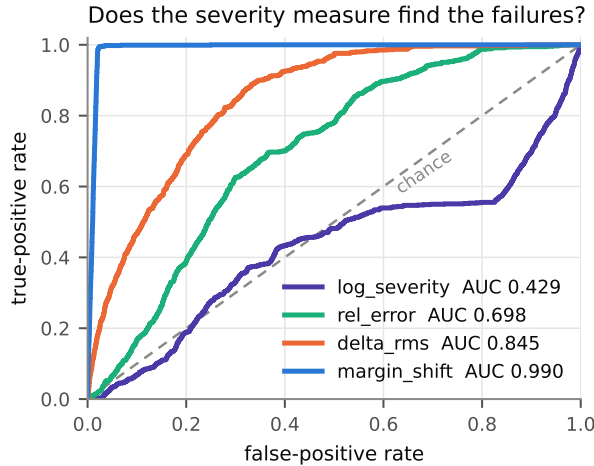


Figure 5: How well each severity measure ranks the faults that actually failed. The measure proposed in the original plan falls below the chance diagonal; margin_shift reaches AUC 0.990.

6.3 Value-aware sampling: 57–120 times fewer injections

A better ranking is worth having only if it saves injections. I tested this offline by replaying sampling strategies against both exhaustive campaigns. The fault space is scored with margin_shift , which is cheap because it needs only gradients and no injections, and divided into six strata at its 50th, 75th, 90th, 97th and 99.5th percentiles. A fifth of the budget goes to a pilot, and the rest is allocated by Neyman allocation [16]. The per-inference rate is a mean rather than a proportion, so each stratum’s variance is estimated from the pilot rather than assumed to be $p(1-p)$.

The target is heavily zero-inflated: 89.2% of ResNet8’s e3m2 faults corrupt no inference at all. The top percentiles of the score isolate the thin tail that holds nearly all the variance. At equal budget, the estimator’s variance falls $57\text{--}120\times$ against uniform sampling on both formats, with no measurable bias at any budget. A 3000-injection stratified campaign is as precise as a uniform campaign of 171,000 (e4m3) to 346,000 (e3m2) injections.

6.4 The plan’s two open design choices

Single-bit injection is sufficient. The plan adopted single-bit flips and left its multi-bit model (the “H-model”) as an extension. To settle the question, I sampled 2000 stored words per format on each of two ResNet8 networks and injected *every* single- and double-bit pattern of each word, 390,000 injections in all. A double flip corrupts $1.04\text{--}1.55\times$ as many inferences as a single flip, and an adjacent pair, the pattern a multi-cell upset produces, $0.85\text{--}1.13\times$. The format ordering and scale dominance are unchanged. Double flips are sub-additive: they do 52–85% of the damage of their two single flips combined, because a second flip can move a word off the NaN code its partner would have reached. Under independent bit flips, doubles within a word reach 1% of the expected damage only at a per-bit flip probability of $2\text{--}3 \times 10^{-3}$. At that rate ResNet8 already holds more than a thousand faulty bits at once, so the one-fault-per-inference assumption fails through accumulation across words long before multiplicity within a word matters.

Enumerate impact exactly; do not learn it. The plan also asked whether impact should be characterised as TreeFI does, by value intervals that a regression tree learns from a pilot campaign, or by exact enumeration over the MX code table. Replayed against both exhaustive campaigns at equal budgets, strata built from the exactly enumerated $|v' - v|$ alone, with no gradient and no injections, cut variance $48\text{--}65\times$ against uniform sampling. Strata learned from a 20% pilot never beat uniform sampling ($0.10\text{--}0.98\times$). Two things defeat the learner. A pilot of a few hundred faults, on a target where 89% of faults do nothing, sees too few failures to place the strata. And in e4m3 the damaging tail is not a value range at all, but the discrete set of (code, bit) pairs that land on NaN, which a code table lists exactly. Interval learning answers a problem that FP32 has, an alphabet of 2^{32} values, and that MX does not have.

6.5 The OCP scale rule distorts block-size trends

Because the OCP rule can clip the block maximum, and smaller blocks have more maxima to clip, the rule interacts with block size. Injecting an identical set of faults at every block size, the element-side any-image SDC varied by $2.2\times$ on ResNet8 and $7.3\times$ on RepVGG under the OCP rule, against $1.04\times$ and $1.5\times$ under the non-clipping rule. Per inference the two rules differ much less ($1.24\times$), and both show the genuine trend of Table 8. Block-size studies should still run a non-clipping control, since the OCP rule does distort the trend it is measured under.

7 Design guidance

The results translate into concrete recommendations for designers of MX accelerators and for anyone evaluating their reliability.

1. **Protect the shared scale first.** A scale fault typically does $13\times$ to more than $1000\times$ the damage of an element fault, in weights and activations alike (less only for e5m2, whose element faults are already NaN-dominated), and the scale is only 2–11% of storage. Parity or ECC on scale bytes is the cheapest protection with the largest payoff. If only some scale bits can be protected, protect those normally at 0, whose flips grow the block.
2. **Neutralise special codes.** Prefer element formats without NaN/Inf codes, or decode special element codes as zero on read. The guard removes most of the element vulnerability of e4m3 and e5m2 on the larger models.
3. **Do not judge reliability by accuracy or bit width.** Formats with equal accuracy differ by orders of magnitude, and among NaN-free formats the ranking depends on the network and task, not on mantissa width.
4. **Treat block size as a trade-off, not a free parameter.** Larger blocks save scale storage but make each scale fault more damaging. With protected scales, large blocks are safe.
5. **Prioritise early layers.** The small stem layer is the most fragile; protection should follow vulnerability, not storage size.

For fault-injection campaigns on MX models:

6. Report the per-inference rate, not the any-image SDC rate.
7. Inject single-bit faults; double-bit faults add no first-order information.
8. Stratify by exact per-code enumeration of the decoded change, weighted by gradient sensitivity where available, for $57\text{--}120\times$ fewer injections at the same precision.

9. Tie every campaign to its exact checkpoint, and verify it by replaying recorded faults.

8 Limitations and threats to validity

ImageNet and DeiT were not evaluated. The original plan included DeiT-Tiny and DeiT-Small on ImageNet. ImageNet was not reachable from any machine available to this project, so CIFAR-100 was used as a stand-in, with a small vision transformer trained from scratch at DeiT-Tiny width. That transformer reaches only 47–49% top-1 on CIFAR-100, far below what a pretrained DeiT achieves on ImageNet. The CIFAR-100 study therefore tests whether the conclusions survive a harder task with smaller margins, and they do; it is not a measurement of DeiT on ImageNet. The trend it reveals, a narrowing gap between formats as margins shrink, suggests that the absolute gaps on ImageNet-scale models may be smaller than on CIFAR-10. The directions of the effects, which held on every network tested, are the more transferable result.

Model scale. The largest network has 7 million parameters. Large language models, listed as optional in the plan, were not studied. The capacity results suggest that larger models absorb perturbations better but not NaNs, which would make special codes even more dominant, but this was not tested directly.

Fault model. Only faults in stored values are modelled: weights, activations and their scales. Faults in the datapath, accumulators, control logic or address decoding are outside the scope, as are timing faults. Every stored bit is assumed equally likely to flip; real memories may have position-dependent rates, which would reweight but not change the per-bit damage measured here.

Evaluation subset. Campaigns score a fixed subset of 200 images (100 for activations) to keep millions of injections affordable. The per-inference rate is robust to this choice, as Section 6.1 shows, but absolute rates carry the sampling uncertainty of the subset, which is reflected in the intervals.

Variation between trained networks. Across ten seeds per width, the spread between independently trained networks is about ten times the uncertainty contributed by 3000 injections. A reliability figure quoted from one trained network is much less certain than its injection count suggests. Key results were therefore replicated over three to ten seeds.

Other. The NaN guard was evaluated in software, not in hardware. The mechanism of the block-size trend for element faults is not established. MXINT8, listed as optional in the plan, was not studied.

9 Conclusion

This project set out to characterise how microscaling formats behave under hardware bit faults, and all five objectives were met. A bit-accurate fault-injection framework, validated against exhaustive ground truth, measured fault sensitivity across formats, block sizes, tensor types and fault locations on convolutional, re-parameterised and Transformer models.

The central hypothesis of the project is confirmed on every network tested. Reliability varies by orders of magnitude between MX formats of equal accuracy, and faults in the shared scale behave very differently from faults in individual elements: they are always more damaging, typically by $13\times$ to more than $1000\times$. The study also explains why. Most element damage comes from the small fraction of flips that land on NaN or infinity codes, which no amount of model capacity can absorb and which a simple read-side guard removes. The rest is governed by the decision margins of the network, not by mantissa width.

Along the way, the study found that the conventional SDC metric can hide the very differences it is meant to reveal, replaced the severity measure of the original plan with one that predicts failure and cuts campaign cost by two orders of magnitude, and settled both of the plan’s open design choices. The most useful next step is to repeat the key campaigns on a pretrained transformer at ImageNet scale, which this project could not reach.

Code and data. The `mxfi` library, every experiment script, the trained networks and the raw campaign outputs are in the project repository (https://github.com/peeyushagarwal2004/CS496_UGP). A detailed log of every finding, including those later revised or withdrawn, is in `docs/findings.md`.

References

- [1] B. D. Rouhani et al., “Microscaling data formats for deep learning,” *arXiv:2310.10537*, 2023.
- [2] Open Compute Project, “OCP Microscaling Formats (MX) Specification, Version 1.0,” 2023.
- [3] G. Li, S. K. S. Hari, M. Sullivan, T. Tsai, K. Pattabiraman, J. Emer, and S. W. Keckler, “Understanding error propagation in deep learning neural network (DNN) accelerators and applications,” in *Proc. SC*, 2017.
- [4] B. Reagen et al., “Ares: A framework for quantifying the resilience of deep neural networks,” in *Proc. DAC*, 2018.
- [5] A. Mahmoud et al., “PyTorchFI: A runtime perturbation tool for DNNs,” in *Proc. DSN Workshops*, 2020.
- [6] S. Hong, P. Frigo, Y. Kaya, C. Giuffrida, and T. Dumitras, “Terminal brain damage: Exposing the graceless degradation in deep neural networks under hardware fault attacks,” in *Proc. USENIX Security*, 2019.
- [7] A. S. Rakin, Z. He, and D. Fan, “Bit-flip attack: Crushing neural network with progressive bit search,” in *Proc. ICCV*, 2019.
- [8] R. Leveugle, A. Calvez, P. Maistri, and P. Vanhauwaert, “Statistical fault injection: Quantified error and confidence,” in *Proc. DATE*, 2009.
- [9] TreeFI: value-aware statistical fault injection for DNNs, in *Proc. ICCAD*, 2026.
- [10] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. CVPR*, 2016.
- [11] C. Banbury et al., “MLPerf Tiny benchmark,” in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.
- [12] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, “RepVGG: Making VGG-style ConvNets great again,” in *Proc. CVPR*, 2021.
- [13] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. ICLR*, 2021.
- [14] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training data-efficient image transformers & distillation through attention,” in *Proc. ICML*, 2021.
- [15] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. American Statistical Association*, vol. 22, 1927.
- [16] J. Neyman, “On the two different aspects of the representative method,” *J. Royal Statistical Society*, vol. 97, 1934.